# HEAS: A HIERARCHICAL EVOLUTIONARY AGENT-BASED SIMULATION FRAMEWORK FOR MULTI-OBJECTIVE POLICY SEARCH

Draft Authors

March 6, 2026

**Abstract**

Agent-based frameworks provide no native support for the pipeline that policy research demands: multi-objective evolutionary search, multi-scenario tournament comparison, and statistically rigorous multi-run validation. Researchers bridge this gap with bespoke coupling code—100–200 lines per project—that is non-reusable and prone to silent metric divergence. HEAS (Hierarchical Evolutionary Agent-based Simulation) eliminates this overhead through four architectural commitments: a Layer/Stream/Arena composition model; a uniform metric contract consumed identically by the evolutionary algorithm, tournament scorer, and bootstrap CI pipeline; native NSGA-II integration with Pareto front output and hypervolume tracking; and a Tournament primitive with four voting rules, sample complexity certification, and noise stability analysis. Against a competent Mesa 3.3.1 reimplementation of an ecological model, HEAS requires 5 lines of coupling code versus 160 (97% reduction). Two case studies validated across 30 independent runs confirm framework correctness and reproducibility.

## 1 INTRODUCTION

160 lines of coupling code—that is the overhead a competent Mesa programmer writes to connect a predator-prey simulation to NSGA-II, tournament comparison, and bootstrap confidence intervals. HEAS reduces this to five lines. The rest of this paper explains why the gap is structural, not stylistic.

Agent-based models (ABMs) have become a primary tool for studying emergent behavior in complex adaptive systems—from ecological communities and financial markets to public health interventions and regulatory

policy design. Using an ABM for *policy search*—finding the configuration of agent parameters or institutional rules that optimizes a multi-dimensional objective—requires infrastructure that no general-purpose ABM framework currently provides.

### 1.1 The Coupling Code Problem

Consider a researcher using Mesa to study regulatory policy in a multi-firm economy. She wants to (1) evolve a Pareto-optimal tax regime using NSGA-II, (2) compare the evolved regime against a reference policy across 32 independent economic scenarios using tournament voting, and (3) report 30-run bootstrap confidence intervals to establish statistical credibility. Each step reads the same welfare metric from the same model, yet in Mesa each step is an independent code path:

- The **DEAP fitness function** extracts welfare from the model's `DataCollector` after each run.
- The **tournament scorer** extracts welfare from a separate episode-evaluation function—possibly with different aggregation (mean vs. final value).
- The **bootstrap CI loop** extracts welfare from a third data collection loop.

When the researcher adds a second objective—welfare inequality—she must edit the DataCollector reporter, the DEAP fitness function, the tournament scorer, and the CI loop, keeping all four in sync manually. The framework provides no enforcement. A silent divergence—the EA optimizing mean welfare while the tournament evaluates final-step welfare—can persist undetected for weeks and invalidate a publication.

This pattern, which we call *coupling code*, appears in every ABM-based policy search project we are aware of [11, 4]. It is not a failure of Mesa or NetLogo—these frameworks were designed for exploratory, single-run agent modeling. But as ABMs are increasingly deployed for optimization and evaluation tasks, the coupling code burden has become a significant source of research friction and reproducibility failures.

### 1.2 What Existing Frameworks Provide

Mesa 3.x [9] provides `DataCollector` for per-step logging, `batch_run` for parameter sweeps, and `SolaraViz` for visualization. It has no native support

for multi-objective EA, Pareto front output, tournament voting, or bootstrap CI. NetLogo's [12] BehaviorSpace supports discrete grid sweeps, not gradient-free optimization over continuous gene spaces. Repast4Py [5] targets HPC distributed ABM but provides no EA integration or tournament evaluation. ABIDES [3] is domain-specific to financial markets. External optimization libraries (PyGMO [2], Optuna [1]) accept Python callables but require the researcher to write the coupling for each new project.

### 1.3 Contributions

We present **HEAS** (Hierarchical Evolutionary Agent-based Simulation), a Python framework that addresses the coupling code problem through four architectural commitments:

**C1—Layered Composition.** The `Layer`/`Stream`/`Arena` triad formalizes multi-level hierarchy as a first-class abstraction. Layers declare their own state and step logic; Streams declare how they couple. Hierarchy is visible to the framework, enabling independent testing and structural reuse across studies.

**C2—Uniform Metric Contract.** Every `Layer` implements `metrics_episode()`, returning a `dict[str, float]` of namespaced keys (e.g., `agg.mean_biomass`). This dictionary is the *single source of truth* consumed identically by the EA fitness function, tournament scorer, and statistical CI pipeline. Adding a metric is a one-line change in one file, propagating to all consumers with no additional synchronization.

**C3—Native Multi-Objective EA.** `run_optimization_simple()` wraps DEAP's [8] NSGA-II [6] with per-run seed management, parallel episode evaluation, Pareto front output, and hypervolume tracking. The researcher writes only the objective function; no DEAP knowledge is required.

**C4—Tournament Evaluation.** The `Tournament` class implements four voting rules (argmax, majority, Borda, Copeland), sample complexity certification, and noise stability analysis as framework-level primitives. Multi-scenario policy comparison becomes a reproducible, statistically certified operation rather than a bespoke script.

Section 5 reports a direct LOC comparison confirming 97% coupling code reduction versus a competent Mesa 3.3.1 implementation.

### 1.4 Paper Organization

Section 2 surveys related work. Section 3 describes the HEAS architecture. Section 4 presents two case studies demonstrating composability across structurally different domains. Section 5 reports evaluation. Section 6 concludes.

## 2 RELATED WORK

### 2.1 Agent-Based Simulation Frameworks

**Mesa** [9] is the dominant Python ABM framework, providing grid and network spaces, agent scheduling, `DataCollector` for per-step metric logging, and `SolaraViz` for browser-based visualization. Mesa is the right tool for exploratory, spatially-oriented single-run modeling. Its `DataCollector` is a pull-based logger: reporters are declared at initialization and queried at each step, producing a time-series DataFrame. This design couples metric collection to the model's internal state; each downstream consumer must write its own extraction function with no framework mechanism to verify consistency. HEAS complements rather than replaces Mesa—for spatial ABMs with rich agent interactions and visualization needs, Mesa remains the preferred choice.

**NetLogo** [12] provides BehaviorSpace for parameter sweeps over a discrete grid. BehaviorSpace does not support continuous gradient-free optimization, produces no Pareto front, and cannot be used for multi-scenario tournament comparison. Its Java runtime is not directly composable with Python statistical pipelines.

**Repast4Py** [5] targets high-performance distributed ABM on HPC systems. Repast's contribution is scalability; HEAS's contribution is composability—they address orthogonal concerns.

**ABIDES** [3] provides a financial market simulation framework with message-passing agents calibrated to real exchange data. It is domain-specific and not designed for policy parameter optimization or multi-scenario comparison.

### 2.2 Optimization over Simulations

**PyGMO/Pagmo** [2] provides high-performance multi-objective optimization accepting any Python callable as the objective. Integration with an ABM must be written by hand per project—reproducing the coupling code

Table 1: Framework capability comparison. ✓ = native; ✗ = not provided; — = not applicable.

| Framework | Hier. | Metric | EA | Tourn. | CI |
|---|---|---|---|---|---|
| Mesa 3.x | ✗ | ✗ | ✗ | ✗ | ✗ |
| NetLogo | ✗ | ✗ | ✗ | ✗ | ✗ |
| Repast4Py | ✗ | ✗ | ✗ | ✗ | ✗ |
| ABIDES | ✗ | ✗ | ✗ | ✗ | ✗ |
| PyGMO + Mesa | — | ✗ | ✓ | ✗ | ✗ |
| **HEAS** | ✓ | ✓ | ✓ | ✓ | ✓ |

that HEAS eliminates. **Optuna** [1] supports simulation objectives but is designed for single-objective Bayesian search, not multi-objective Pareto search with tournament evaluation. **DEAP** [8], which HEAS uses internally, provides NSGA-II but requires the user to wire all infrastructure. HEAS wraps DEAP with per-run seeding, parallel evaluation, and hypervolume tracking, exposing a single-call interface that requires no DEAP knowledge.

### 2.3 Policy Search in Agent-Based Models

Domain-specific works have combined ABMs with evolutionary search. Pangallo et al. [11] apply genetic algorithms to a macroeconomic ABM using custom fitness extraction—a representative example of the coupling code pattern HEAS eliminates. Zheng et al. [13] learn tax policies in the AI Economist using deep RL; this requires a differentiable simulation. HEAS targets the complementary setting: stochastic, non-differentiable, episodic simulation where gradient-free Pareto search is appropriate. Caprì et al. [4] evaluate transportation policies using NetLogo with external Python scripts, manually maintaining metric consistency—the exact divergence risk that HEAS's metric contract prevents.

### 2.4 Gap Summary

Table 1 summarizes the capability gap that HEAS addresses. HEAS occupies the intersection of these lines of work: an ABM runtime with layered hierarchy, connected to evolutionary search via a uniform metric contract, augmented with tournament evaluation and statistical CI as framework-level primitives—a combination not available in any existing tool.

# 3 THE HEAS FRAMEWORK

HEAS is implemented in Python 3.11 and depends on DEAP for evolutionary algorithms, NumPy for numerical computation, and SciPy for statistical utilities. All components—EA algorithm, voting rule, statistical estimator—can be replaced without changing simulation or metric contract code.

## 3.1 Layered Composition

The fundamental abstraction is the `Layer`, a unit of simulation logic with a four-method contract:

```
class Layer:
    def step(self) -> None:
        """Advance state by one discrete timestep."""
    def metrics_step(self) -> dict[str, float]:
        """Per-timestep metrics: fed to dashboards."""
    def metrics_episode(self) -> dict[str, float]:
        """End-of-episode aggregates: fed to EA,
           tournament, and CI pipeline."""
    def reset(self) -> None:
        """Reset to initial conditions."""
```

The contract separates *what a layer computes* from *how its outputs are used*. Any downstream component reads the same `metrics_episode()` dictionary using the same key string. This is the metric contract: a single source of truth enforced by the interface, not by convention.

Layers compose into **Streams**—named groups of layers forming a coherent sub-simulation—and Streams compose into an **Arena**, which orchestrates step ordering, inter-stream signal routing, and episode-level aggregation. The naming convention `stream_name.metric_name` (e.g., `agg.mean_biomass`) ensures globally unambiguous keys regardless of Arena size.

## 3.2 Why Hierarchy Matters

Existing frameworks represent agents as flat objects sharing a global model state. When a researcher adds climate forcing, spatial fragmentation, and an aggregation layer to a predator-prey model, these appear as attributes of the Mesa `Model` class—invisible to the framework and impossible to reuse, test in isolation, or compose into a new study.

In HEAS, the same model is a five-stream Arena:

```
EcoArena
```

```
+-- ClimateStream   (temperature, shock events)
+-- LandscapeStream (patch quality, fragmentation)
+-- PreyStream      (logistic growth, dispersal)
+-- PredStream      (predation, mortality)
+-- AggStream
     -> agg.mean_biomass, agg.cv, agg.extinct
```

Each stream can be developed, validated, and replaced independently. Swapping the climate model requires only modifying `ClimateStream.step()`—the EA, tournament, and AggStream see no change because they interact only through the metric contract.

### 3.3 Native Multi-Objective EA Integration

`run_optimization_simple()` connects DEAP's NSGA-II to any objective defined via the metric contract:

```
result = run_optimization_simple(
    objective_fn=eco.trait_objective,
    n_genes=2,
    gene_bounds=[(0.0, 1.0), (0.0, 1.0)],
    pop_size=20, n_generations=10, seed=42,
)
# result["hof_fitness"] -- Pareto front
# result["hv"]          -- hypervolume
# result["log"]         -- per-generation stats
```

The wrapper handles DEAP's creator/toolbox registration, crossover and mutation wiring, hall-of-fame management, and hypervolume computation. A researcher adds NSGA-II to a new simulation by writing only the objective function (typically 5–10 lines reading from `run_many()` results) and calling `run_optimization_simple()`. The EA algorithm is itself replaceable: `run_ea()` also accepts CMA-ES and single-objective hillclimbing. Section 5.4 shows that simple hillclimbing outperforms NSGA-II on a 2-gene landscape—confirming that algorithm-agnosticism is not only an architectural claim but a practical benefit.

`run_many()` executes $n$ episodes in parallel via `ProcessPoolExecutor`, with per-episode seeds derived deterministically from a base seed. The same base seed always yields the same evaluation result, making individual fitness calls reproducible.

### 3.4 Tournament Evaluation

Comparing candidate policies is formalized as a `Tournament`: a cross-product of participants and scenarios, evaluated for $n_{\text{ep}}$ episodes each, aggregated by one of four voting rules.

**Argmax**: participant with the highest total score across all episodes. **Majority voting**: pairwise—the participant winning more than half the episodes in each pair advances; most pairwise wins overall. **Borda count**: each participant receives points equal to the number of opponents beaten per episode; totals summed. **Copeland**: pairwise win/loss matrix; participant with most net wins.

HEAS reports the *agreement matrix* (fraction of (scenario, repeat) pairs where each pair of voting rules selects the same winner), the *sample complexity curve* $P(\text{correct})$ as a function of $n_{\text{ep}}$, and a Kendall's $\tau$ [10] vs. noise $\sigma$ stability curve.

### 3.5 Statistical Infrastructure

`heas/utils/stats.py` provides: `summarize_runs()` (mean, std, median, BCa bootstrap 95% CI [7]), `wilcoxon_test()`, `cohens_d()`, and `kendall_tau()` with bootstrap CI. `heas/utils/pareto.py` provides `hypervolume()` [14] and `auto_reference_point()`. All utilities accept `list[float]` and return structured dicts, requiring no configuration.

## 4 CASE STUDIES

We demonstrate HEAS's composability on two case studies that differ structurally: an ecological population model with stochastic spatial dynamics and a multi-firm economic regulation model with hierarchical governance. Neither model is proposed as a validated scientific account of its domain—both serve as *composition vehicles* whose structural diversity validates the generality of the Layer/Stream/Arena abstraction.

### 4.1 Ecological Population Management

The ecological model represents a fragmented landscape with prey and predator populations interacting across climate-driven patch dynamics. It is implemented as a five-stream Arena:

```
EcoArena (200 steps/episode)
 +-- ClimateStream   -- temperature, shock_prob
```

```
+-- LandscapeStream -- patch quality Q in [0,1]
+-- PreyStream      -- logistic growth:
      dx = r*Q*x*(1-x/K) - loss(risk,x,y)
+-- PredStream      -- dy = conv*x*y - mort*y
+-- AggStream
      -> agg.mean_biomass (mean of x+y)
      -> agg.cv           (coefficient of variation)
      -> agg.extinct      (predator extinction flag)
```

The policy space is two-dimensional: **risk** $\in [0, 1]$ and **dispersal** $\in [0, 1]$. The carrying capacity $K = 1{,}000$ ensures genuine predator-prey dynamics (breakeven prey $= \text{mort}/(\text{conv} \cdot 0.01) = 750 < K$). At $K < 750$, the predator goes extinct in every episode, making the extinction objective degenerate; detecting this required observing HV std $= 0.000$ across 30 runs—a diagnostic only available via the framework's reproducibility infrastructure.

NSGA-II minimizes $\{-$`agg.mean_biomass`, `agg.cv`$\}$ (maximize biomass, minimize variability). Tournament evaluation uses 8 scenarios crossing fragmentation $\in \{0.2, 0.6\}$ and shock_prob $\in \{0.05, 0.15, 0.20, 0.30\}$.

### 4.2 Enterprise Regulatory Design

The enterprise model represents a regulated economy with firms, a sector regulator, and a government. It is implemented as a four-layer Arena:

```
EnterpriseArena
 +-- FirmLayer      -- production, compliance cost
 +-- RegulatorLayer -- audit, sector coordination
 +-- GovernmentLayer -- tax, subsidy allocation
 +-- WelfareLayer
       -> welfare.total, welfare.gini,
          welfare.compliance_rate
```

The four-dimensional policy space covers tax_rate, audit_intensity, subsidy, and penalty_rate. NSGA-II minimizes $\{-$`welfare.total`, `welfare.gini`$\}$. The 32-scenario evaluation grid crosses governance regime, demand shock, audit frequency, firm count, and cost structure (2 levels each).

### 4.3 Structural Comparison

Table 2 shows that the two case studies differ along every structural axis. The metric contract (`metrics_episode()` $\rightarrow$ `dict[str, float]`) generalizes from population dynamics to economic welfare without any change in framework code.

Table 2: Structural comparison of case studies.

| Property | Ecological | Enterprise |
|---|---|---|
| Hierarchy units | 5 streams | 4 layers |
| Policy genes | 2 | 4 |
| Objectives | biomass, CV | welfare, Gini |
| Scenarios | 8 | 32 |
| Episode length | 200 steps | 50 steps |
| Key dynamic | stochastic | deterministic |

# 5 EVALUATION

We evaluate HEAS along four dimensions: coupling code reduction versus Mesa (the primary framework comparison), reproducibility of multi-run studies, tournament reliability under voting rule variation and measurement noise, and algorithm agnosticism across optimizer choices.

## 5.1 Mesa Comparison: Coupling Code Reduction

We implement the ecological case study as a competent Mesa 3.3.1 model (`experiments/mesa_eco.py`) using all Mesa best practices: `DataCollector` for metric logging, a separate episode runner function, `ProcessPoolExecutor` for parallel evaluation, and individual fitness and tournament scoring functions. *This is not a strawman—it is the implementation a Mesa-proficient researcher would produce.* We count non-blank, non-comment lines in each component, separating model logic (identical between implementations) from coupling code (lines that exist solely to wire simulation output to a downstream consumer).

The HEAS column's five lines correspond to one call to `run_many()` in the objective function body and four lines reading metric keys from the returned dict. All other functionality—parallelism, seeding, Pareto tracking, tournament voting, bootstrap CI—is provided by the framework at zero additional LOC.

**Objective extension cost** (Table 4): adding a second objective (biomass coefficient of variation) requires editing three files and ~6 lines in Mesa, with no framework mechanism to verify consistency. HEAS requires one file and one line, with a structural consistency guarantee: all consumers read the same `dict` key.

**Metric divergence risk**: in Mesa, a developer who changes the EA objective from `final_prey` to `mean_biomass` but does not update the tour-

Table 3: Coupling code LOC: Mesa 3.3.1 vs. HEAS. Model logic lines are excluded as they are identical in both.

| Task | Mesa | HEAS | Saved |
|---|---|---|---|
| Metric contract / DataCollector setup | 15 | 0 | 15 |
| Episode metric extraction function | 20 | 0 | 20 |
| Multi-episode sequential runner | 22 | 1 | 21 |
| Parallel episode runner | 20 | 0 | 20 |
| EA fitness function glue | 14 | 3 | 11 |
| Tournament scoring function | 18 | 0 | 18 |
| Per-run seed management (30 runs) | 12 | 0 | 12 |
| Bootstrap CI computation | 35 | 0 | 35 |
| Adding a second objective | 4 | 1 | 3 |
| **Total** | **160** | **5** | **155 (97%)** |

Table 4: Cost of adding a second optimization objective.

| | Mesa | HEAS |
|---|---|---|
| Files edited | 3 | 1 |
| Lines added | 6 | 1 |
| Consistency guarantee | None | Structural |

nament scorer will silently obtain results where the optimization target and evaluation criterion differ. In HEAS, this is structurally impossible: both the EA and the tournament read `ep_metrics["agg.mean_biomass"]` from the same `metrics_episode()` return value.

**Parallelism**: `n_jobs` to `run_many()` is a zero-LOC change in HEAS; Mesa requires ˜20 lines of `ProcessPoolExecutor` boilerplate per project. For the lightweight ODE simulation used here (≈0.004 s/episode), process startup overhead (≈9 s) dominates and `n_jobs=4` provides no runtime speedup at small episode counts. For computationally heavier agents (>0.1 s/episode), `n_jobs` delivers linear speedup. The LOC ergonomics benefit is unconditional.

## 5.2 Reproducibility: 30-Run Statistical Studies

We run 30 independent NSGA-II experiments per case study using framework-managed per-run seeding and report hypervolume (HV) with 95% BCa bootstrap CIs via `summarize_runs()`. Table 5 shows the results.

The ecological HV distribution is bimodal: runs settle near local optima (HV ≈ 4–6.5) or reach the global Pareto front (HV ≈ 11.7–11.8). The CI width

Table 5: Hypervolume statistics across 30 independent NSGA-II runs. Reference point: auto with 10% margin. 95% BCa bootstrap CI.

| Study | HV mean | HV std | 95% CI | $n$ |
|---|---|---|---|---|
| Ecological | 7.665 | 3.518 | $[6.424, 8.914]$ | 30 |
| Enterprise | 4317.5 | 19.4 | $[4311.2, 4326.0]$ | 30 |

Table 6: Voting rule agreement matrix (fraction of trials, $n = 30 \times 8 = 240$).

| | Argmax | Majority | Borda | Copeland |
|---|---|---|---|---|
| Argmax | 1.000 | 1.000 | 1.000 | 1.000 |
| Majority | — | 1.000 | 1.000 | 1.000 |
| Borda | — | — | 1.000 | 1.000 |
| Copeland | — | — | — | 1.000 |

of 2.49 units correctly captures this spread. The enterprise HV variance is narrow ($\pm$19.4) because the enterprise landscape is more unimodal.

Crucially, the K= 1,000 parameter selection was informed by this reproducibility infrastructure: at $K = 120$ (the original parameterization), the predator went extinct in every episode, reducing both objectives to degenerate constants and producing HV std = 0.000 across all 30 runs. The degeneracy was detected by observing the collapsed CI; a single-run Mesa study would have reported one number without revealing the degenerate distribution.

### 5.3 Tournament Validation

**Voting rule agreement** (Table 6): we run 30 repeats of the 8-scenario tournament with 100 episodes per scenario. Participants are: champion (risk = 0.003, dispersal = 0.959, evolved by NSGA-II), reference (risk = 0.55, dispersal = 0.35, paper baseline), and contrarian (risk = 0.85, dispersal = 0.15, high-risk/low-dispersal). The champion's mean biomass ($\approx$940) exceeds the reference ($\approx$785) by a margin of $\approx$155 units. All four voting rules agree in 100% of (scenario, repeat) pairs.

**Noise stability** (Table 7): we inject Gaussian noise $\mathcal{N}(0, \sigma^2)$ to episode scores and measure Kendall's $\tau$ between the clean and noisy ranking. Rankings are stable ($\tau = 1.0$) for noise up to 0.65% of the inter-policy margin, and degrade gracefully beyond. Realistic simulation measurement variance is far below 10% of typical inter-policy margins, placing well-designed tournaments in the $\tau > 0.94$ regime.

Table 7: Ranking stability under score perturbation ($n = 30$ repeats, 50 eps/scenario). $\sigma$/margin: noise relative to inter-policy score gap (≈155).

| $\sigma$ | $\sigma$/margin | Mean $\tau$ | 95% CI |
|---|---|---|---|
| 0 | 0.0% | 1.000 | $[1.000, 1.000]$ |
| 1 | 0.65% | 1.000 | $[1.000, 1.000]$ |
| 10 | 6.5% | 0.944 | $[0.922, 0.967]$ |
| 50 | 32% | 0.744 | $[0.700, 0.789]$ |
| 100 | 65% | 0.619 | $[0.567, 0.675]$ |
| 200 | 130% | 0.508 | $[0.453, 0.567]$ |

Table 8: Algorithm ablation on ecological 2-gene landscape ($n = 10$ runs, pop = 20, ngen = 10).

| Strategy | HV mean | HV std | 95% CI |
|---|---|---|---|
| Simple (hillclimb) | 19.66 | 6.84 | $[14.82, 22.92]$ |
| NSGA-II | 9.99 | 6.84 | $[6.67, 14.81]$ |
| Random grid | 7.61 | 0.33 | $[7.42, 7.81]$ |

### 5.4 Algorithm Agnosticism

To validate HEAS's optimizer-agnosticism claim, we run three strategies on the ecological policy space (Table 8). Simple hillclimbing outperforms NSGA-II on this 2-gene landscape: the Pareto front is effectively one-dimensional (dominated by the dispersal gene), so single-objective search concentrates population on the biomass-maximizing direction while NSGA-II's diversity maintenance distributes population across the CV trade-off. This result reinforces rather than undermines the framework claim: HEAS allows the researcher to select the algorithm appropriate to the landscape complexity without any framework changes.

## 6 CONCLUSION

HEAS addresses the coupling code problem in ABM-based policy search through four architectural contributions: the Layer/Stream/Arena composition model, the uniform metric contract, native NSGA-II integration, and the Tournament evaluation primitive. Direct comparison with Mesa 3.3.1 confirms 97% coupling code reduction (5 vs. 160 lines); adding a second objective requires one line in one file in HEAS versus six lines across three files in Mesa, with HEAS providing structural consistency guarantees that Mesa

cannot. Two case studies validated across 30 independent runs each confirm framework correctness and reproducibility. Tournament evaluation with four voting rules shows 100% agreement under well-separated participants; $\tau > 0.94$ at realistic noise levels confirms graceful degradation.

**Limitations.** For lightweight ODE simulations (<0.01 s/episode), process startup overhead (≈9 s) dominates and `n_jobs` provides no runtime benefit; use `n_jobs=1`. HEAS is designed for ODE-style and tabular agent simulations, not spatial grid simulations; for spatially-explicit ABMs with rich movement and neighborhood interaction, Mesa remains the preferred choice.

**Future work.** The highest-value near-term extension is a `MesaLayerAdapter` that wraps a Mesa `Model` as a HEAS `Layer`, exposing `DataCollector` reporters as `metrics_episode()` keys. This would make HEAS's EA and tournament primitives available to existing Mesa models without rewriting the simulation code. Additional planned work includes a Ray/Dask backend for HPC-scale parallel evaluation, automated scenario generation via Latin Hypercube Sampling, and adaptive hypervolume reference point estimation.

HEAS is available as an open-source Python package at `https://github.com/ryZhangHason/heas`; all experiments in this paper are fully reproducible from the scripts in `experiments/`.

## References


[1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631. ACM, 2019.

[2] Francesco Biscani and Dario Izzo. A parallel global multiobjective framework for optimization: Pagmo. *Journal of Open Source Software*, 5(53):2338, 2020.

[3] David Byrd, Maria Hybinette, and Tucker H. Balch. ABIDES: Towards high-fidelity multi-agent financial market simulation. In *Proceedings of the 2020 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation (PADS)*, pages 11–22. ACM, 2020.

[4] Salvatore Caprì, Maria Ignaccolo, Giuseppe Inturri, Alessandro Pluchino, and Andrea Rapisarda. Evaluating transport policies through NetLogo

multi-agent simulation and evolutionary algorithms. *Transportation Research Procedia*, 69:613–620, 2023.

[5] Nick Collier and Michael J. North. Repast4Py: Python-based large-scale agent-based modeling. In *Proceedings of the 2022 Winter Simulation Conference*, pages 192–203. IEEE Press, 2022.

[6] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002.

[7] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, New York, 1994.

[8] Félix-Antoine Fortin, François-Michel De Rainville, Marc-André Gardner, Marc Parizeau, and Christian Gagné. DEAP: Evolutionary algorithms made easy. *Journal of Machine Learning Research*, 13:2171–2175, 2012.

[9] Jackie Kazil, David Masad, and Andrew Crooks. Utilizing Python for agent-based modeling: The Mesa framework. In *Social, Cultural, and Behavioral Modeling (SBP-BRiMS)*, pages 308–317. Springer, 2020. Mesa is available at `https://github.com/projectmesa/mesa`.

[10] Maurice G. Kendall. A new measure of rank correlation. *Biometrika*, 30(1-2):81–93, 1938.

[11] Marco Pangallo, J. Doyne Farmer, James W. Sanders, and Tobias Galla. Towards a complete classification of the macroeconomic consequences of agent heterogeneity. *Journal of Economic Dynamics and Control*, 107:103716, 2019.

[12] Uri Wilensky. NetLogo. Center for Connected Learning and Computer-Based Modeling, Northwestern University, Evanston, IL, 1999. `http://ccl.northwestern.edu/netlogo/`.

[13] Stephan Zheng, Alexander Trott, Sunil Srinivasa, David C. Parkes, and Richard Socher. The AI economist: Taxation policy design via two-level deep multiagent reinforcement learning. *Science Advances*, 8(18):eabk2607, 2022.

[14] Eckart Zitzler and Lothar Thiele. Multiobjective optimization using evolutionary algorithms—a comparative case study. *Parallel Problem Solving from Nature*, pages 292–301, 1998.